

Persistance des données avec localStorage

Objectifs de cette section

Jusqu'à présent, votre todo list fonctionne... mais oublie tout au rechargement de la page.

Dans cette section, vous allez apprendre à **sauvegarder les données localement** dans le navigateur grâce à `localStorage`.

À la fin de cette section, vous serez capable de :

- Comprendre le fonctionnement du `localStorage`.
- Enregistrer et récupérer des données dans le navigateur.
- Convertir des tableaux en chaînes JSON, et inversement.
- Synchroniser automatiquement l'affichage avec les données stockées.

1. Comprendre le localStorage

Le `localStorage` est un espace de stockage fourni par le navigateur, accessible via JavaScript.

Caractéristiques :

- Il stocke des paires **clé/valeur**.
- Les données sont **persistantes**, même après fermeture du navigateur.
- Il ne stocke que des **chaînes de caractères** → il faut convertir les objets avec `JSON.stringify()` et `JSON.parse()`.

Exemple : stocker et récupérer une tâche

```
// script.js
// Stocker une tâche
localStorage.setItem("task", "Acheter du pain");

// Récupérer la tâche
const task = localStorage.getItem("task");
console.log(task); // "Acheter du pain"
```

2. Enregistrer une liste de tâches

Lorsqu'une tâche est ajoutée, on peut :

- Ajouter la nouvelle valeur à un tableau.
- Convertir ce tableau en chaîne avec `JSON.stringify`.
- L'enregistrer dans `localStorage.setItem("tasks", ...)`.

Et lorsqu'on recharge la page :

- Lire les données avec `localStorage.getItem("tasks")`.
- Les convertir en tableau avec `JSON.parse(...)`.
- Afficher à nouveau chaque tâche.

Exemple : sauvegarder et recharger une liste

```
// script.js
// Exemple d'enregistrement
const tasks = ["Acheter du pain", "Sortir le chien"];
localStorage.setItem("tasks", JSON.stringify(tasks));

// Exemple de récupération
const data = localStorage.getItem("tasks");
const parsedTasks = JSON.parse(data);
console.log(parsedTasks); // ["Acheter du pain", "Sortir le chien"]
```

3. Synchroniser le rendu avec les données

Dès le chargement de la page, il faut :

- Vérifier si des données sont présentes dans le `localStorage`.
- Si oui, les afficher.
- À chaque ajout ou suppression, mettre à jour la source de vérité (le tableau) **et** réécrire dans le `localStorage`.

Exemple : synchronisation automatique

```
// script.js
// Initialisation
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function saveTasks() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}

function renderTasks() {
  const list = document.getElementById("taskList");
  list.innerHTML = "";
  tasks.forEach((task, index) => {
    const li = document.createElement("li");
    li.textContent = task;

    const btn = document.createElement("button");
    btn.textContent = "Supprimer";
    btn.addEventListener("click", () => {
      tasks.splice(index, 1);
      saveTasks();
      renderTasks();
    });

    li.appendChild(btn);
    list.appendChild(li);
  });
}

// Ajout via formulaire
document.getElementById("taskForm").addEventListener("submit", (e) => {
  e.preventDefault();
  const input = document.getElementById("taskInput");
  const value = input.value.trim();
  if (value !== "") {
    tasks.push(value);
    saveTasks();
    renderTasks();
    input.value = "";
  }
});

// Affichage initial
renderTasks();
```

4. Exercice de mise en pratique

Consignes

1. Reprenez votre projet.
2. Lorsque l'utilisateur ajoute une tâche :
 - Ajoutez-la dans un tableau `tasks`.
 - Enregistrez ce tableau dans le `localStorage`.
3. Lors du chargement de la page :
 - Récupérez les données depuis le `localStorage`.
 - Affichez automatiquement toutes les tâches existantes.

4. (Bonus) Lorsqu'une tâche est supprimée, mettez à jour le tableau et le `localStorage` .

Organisation attendue

```
/exercices
└─ 06-localstorage
   ├── index.html
   ├── script.js
   └─ todo.js
```